



Experiment 9

Prepared By:

Name: Mahin Raj

Course: BCA (2nd Semester)

SAP ID: 590025881

Academic Year: 2025-2026

Submitted To:

Mr. Vibhu Gautam

Faculty, School of Computer Science

UPES, Dehradun



mahin-mr12 / python-programming



<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

python-programming / Experiments / Python Exp-9.ipynb



mahin-mr12 commit

9eed2b6 · 2 months ago



384 lines (384 loc) · 8.85 KB

Preview

Code

Blame

Raw



Experiment 9: Classes and objects

1. Create a class of student (name, sap id, marks[phy,chem,maths]). Create 3 objects by taking inputs from the user and display details of all students.

```
In [ ]: class Student:
    def get_data(self):
        self.name = input("Enter Name: ")
        self.sap_id = input("Enter SAP ID: ")
        self.phy = int(input("Enter Physics Marks: "))
        self.chem = int(input("Enter Chemistry Marks: "))
        self.maths = int(input("Enter Maths Marks: "))

    def display(self):
        print("\nName:", self.name)
        print("SAP ID:", self.sap_id)
        print("Physics:", self.phy)
        print("Chemistry:", self.chem)
        print("Maths:", self.maths)

# Create 3 objects
students = []

for i in range(3):
    print(f"\nEnter details of Student {i+1}")
    s = Student()
    s.get_data()
    students.append(s)

print("\n--- Student Details ---")
for s in students:
    s.display()
```

Enter details of Student 1

Enter details of Student 2

Enter details of Student 3

--- Student Details ---

Name: krish
SAP ID: 590020086
Physics: 89
Chemistry: 78
Maths: 86

Name: raj
SAP ID: 5202222333
Physics: 89
Chemistry: 57
Maths: 49

Name: rhul
SAP ID: 7822665526
Physics: 85
Chemistry: 96
Maths: 74

2. Add constructor in the above class to initialize student details of n students and implement following methods:

Display() student details

Find Marks_percentage() of each student

Display result() [Note: if marks in each subject >40% than Pass else Fail]

Write a Function to find average of the class.

```
In [ ]: class Student:
    def __init__(self, name, sap_id, phy, chem, maths):
        self.name = name
        self.sap_id = sap_id
        self.phy = phy
        self.chem = chem
        self.maths = maths

    def display(self):
        print("\nName:", self.name)
        print("SAP ID:", self.sap_id)
        print("Marks:", self.phy, self.chem, self.maths)

    def marks_percentage(self):
        total = self.phy + self.chem + self.maths
        return total / 3
```

```

    return total / n

def result(self):
    if self.phy > 40 and self.chem > 40 and self.maths > 40:
        return "Pass"
    else:
        return "Fail"

# Input n students
n = int(input("Enter number of students: "))
students = []

for i in range(n):
    print(f"\nEnter details of Student {i+1}")
    name = input("Name: ")
    sap = input("SAP ID: ")
    phy = int(input("Physics: "))
    chem = int(input("Chemistry: "))
    maths = int(input("Maths: "))

    students.append(Student(name, sap, phy, chem, maths))

# Display details
total_class = 0

for s in students:
    s.display()
    percent = s.marks_percentage()
    print("Percentage:", percent)
    print("Result:", s.result())
    total_class += percent

# Class average
print("\nClass Average:", total_class / n)

```

Enter details of Student 1

Name: krish
 SAP ID: 590020086
 Marks: 89 76 86
 Percentage: 83.66666666666667
 Result: Pass

Class Average: 83.66666666666667

3. Create programs to implement different types of inheritances.

```
In [ ]: #a)Single Inheritance
class Parent:
    def show(self):
        print("Parent Class")

class Child(Parent):
    def display(self):
        print("Child Class")

c = Child()
c.show()
c.display()
```

Parent Class
Child Class

```
In [ ]: #b)Multilevel Inheritance
class A:
    def showA(self):
        print("Class A")

class B(A):
    def showB(self):
        print("Class B")

class C(B):
    def showC(self):
        print("Class C")

obj = C()
obj.showA()
obj.showB()
obj.showC()
```

Class A
Class B
Class C

```
In [ ]: #c)Multiple Inheritance
class Father:
    def show1(self):
```

```
    def show1(self):
        print("Father")

class Mother:
    def show2(self):
        print("Mother")

class Child(Father, Mother):
    def show3(self):
        print("Child")

c = Child()
c.show1()
c.show2()
c.show3()
```

4. Create a class to implement method Overriding.

```
In [ ]: class Animal:
        def sound(self):
            print("Animal makes sound")

        class Dog(Animal):
            def sound(self):
                print("Dog barks")

d = Dog()
d.sound()
```

Dog barks

5. Create a class for operator overloading which adds two Point Objects where Point has x & y values

e.g. if

P1(x=10,y=20)

P2(x=12,y=15)

P3=P1+P2 => P3(x=22,y=35)

```
In [ ]: class Point:
        def __init__(self, x, y):
            self.x = x
```

```
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def display(self):
        print(f"Point(x={self.x}, y={self.y})")

P1 = Point(10, 20)
P2 = Point(12, 15)

P3 = P1 + P2

P3.display()

Point(x=22, y=35)
```